

Bit Manipulation

1. Fundamentals of Binary Representation

- Base -2 System: ~~base~~ how integers map to binary digits
- Data types in Java: Understand the bit sizes of bytes (8-bit), short ~~(8-bit)~~ (16-bit), int (32-bit) and long (64-bit)

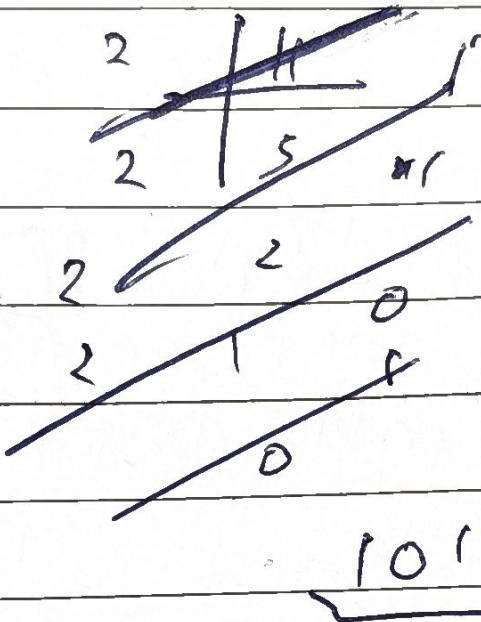
2. Core Bitwise operators

- And (&) : returns 1 if both bits are 1 (Used for masking)
- Or (|) : return 1 if at least one bit is 1. (Used for setting bits)
- XOR (^) : Returns 1 if bits are different, used for toggling bits
and ~~cancel~~ cancelling duplicates
- Not (~) :
flips all bits (one's complement)

[1, 2, 3, 2, 2]

1: 1 2: 1 3: 1

low [1]
0



2	11	1
2	5	1
2	2	0
2	1	1
	0	

(11)₁₀ → (1011)₂

000...1101,
32 bits

000...1011

Most Significant Bit → left side of bit represent (+) (-)
Least Significant Bit → Right most bit (odd / even)

1's Complement = flip digits

2's Complement = +1

000...1011

111110100 ← 1's Complement

+1 ← 2's Complement

111110101

Important to find -n.

Now decimal to Binary:-

$$(1011)_2 = ?$$

$$\boxed{1} \quad \boxed{0} \quad \boxed{1} \quad \boxed{1}$$
$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$8 + 0 + 2 + 1 = \boxed{11}$$

(0) (1)

(+) (-)

Decimal $\rightarrow$ Binary

Binary $\rightarrow$ Decimal

set bit $\rightarrow 1$

unset bit $\rightarrow 0$

① ① ① ①

set unset set set

Range

largest

$$2^3 - 1$$

top

0

1 1 1 1 1 1 1 1 1 1

~~Shortage~~ Shortest)
(-)

$$\begin{array}{r} 1's\ complement \\ 2's\ complement \end{array} \begin{array}{r} 1\ 0\ 0\ 0\ 0\ 0 \\ 0\ 1\ 1\ 1\ 1\ 1 \\ \hline 1\ 0\ 0\ 0\ 0\ 0 \end{array} + 1$$

$$-2^{31} \leftrightarrow 2^{31} - 1$$

$[-2^{31}, 2^{31} - 1]$ range of integers

Bitwise Operator

& $\rightarrow$ AND

| $\rightarrow$ OR

^ $\rightarrow$ XOR

>> $\rightarrow$ Right Shift

>>> $\rightarrow$ Unsigned Right Shift

<< $\rightarrow$ Left Shift

~ $\rightarrow$ NOT

AND (&)

B ₁	OP	B ₂	R
0	&	0	0
0	&	1	0
1	&	0	0
1	&	1	1

11 & 13

11 → 1011

13 → 1101

1001 → 9

2	13	1
2	6	0
2	3	1
2	1	1
	0	

OR (|)

B ₁	OP	B ₂	R
0		0	0
0		1	1
1		0	1
1		1	1

11 | 13 =

11 → 1011

13 → 1100

1111

8 + 4 + 2 + 1 = 15

XOR ($\wedge$)

Bits Different = 1

$11 \wedge 13$

Same = 0

B_1	OP	B_2	R
0	$\wedge$	0	0
0	$\wedge$	1	1
1	$\wedge$	0	1
1	$\wedge$	1	0

$11 \rightarrow 1011$

$13 \rightarrow \begin{array}{r} 1101 \\ \underline{0110} \end{array}$

$2^4 + 2$

$4 + 2 = 6$

5 bit

NOT ($\sim$)

$\rightarrow$ flip bits

$0 \rightarrow 1$

$1 \rightarrow 0$

MSB

~~00111~~
~~01000~~
MSB

$\sim \rightarrow$

00111

11000

00111

+ 1

01000

$\downarrow$

~~00~~ 8

$\downarrow$

-8

$\sim N = (-)_{\text{neg}}$

$\downarrow$
2's

$\sim (-N)$

$\downarrow$

$\sim (2's)$

$\sim N = (+)_{\text{stop}}$

because at first not the number was -ve

$$\sim(-7)$$

$$= \begin{array}{r} 00111 \\ 11000 \\ +1 \\ \hline \end{array}$$

$$\sim(-7) \Rightarrow 11001$$

$$\sim(11001)$$

$$\underbrace{10110}_{\rightarrow} 6$$

Trick

$$\sim(N) = \cancel{N} - (N+1)$$

~~Trick~~

$$\begin{aligned} \sim(7) &= -(7+1) \\ &= -8 \end{aligned}$$

$$\begin{aligned} \sim(-7) &= -(-7+1) \\ &= -(-6) \\ &= 6 \end{aligned}$$

Right Shift Operator ($\gg$)

$$x = 11$$

$$11 \Rightarrow \begin{array}{ccccccc} & 0 & 0 & 0 & & 1 & 0 & 1 & 1 \\ & \underline{} & \underline{} & \underline{} & & \underline{} & \underline{} & \underline{} & \underline{} \\ & & & & & 3 & 2 & 1 & 0 \end{array}$$

$$m \gg 1 \rightarrow 1 \text{ bit at a time} \Rightarrow \frac{m}{2^1}$$

$$m \gg 2 \rightarrow 2 \text{ bit at a time} \Rightarrow \frac{m}{2^2}$$

$$m \gg k = k \text{ bits at a time} \Rightarrow \frac{m}{2^k}$$

Left shift Operator ($\ll$)

$$11 \ll 1$$

$$11 \rightarrow \begin{array}{cccc} 0 & \dots & 1 & 0 & 1 & 1 \\ \hline 31 & & 3 & 2 & 1 & 0 \end{array}$$

$$11 \ll 1 \rightarrow \begin{array}{cccccc} 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ \hline 31 & 4 & 3 & 2 & 1 & 0 & \end{array} \leftarrow \begin{array}{l} \text{0 by default} \end{array}$$

$$11 \ll 1 = 22$$

$$m \ll k = m \times 2^k$$

1) Odd - Even Check

$$\begin{array}{cccc} 1 & 0 & 1 & 0 \\ \hline 3 & 2 & 1 & 0 \end{array} \leftarrow \text{even}$$

$$0 \times 2^1 = 1 \times 0 = 0$$

$$\begin{array}{cccc} 1 & 0 & 1 & 1 \\ \hline 3 & 2 & 1 & 0 \end{array} \leftarrow \text{odd}$$

$$1 \times 2^1 = 1 \times 1 = 1$$

$$(10) \rightarrow \begin{array}{r} 101 \\ \times 000 \\ \hline 000 \end{array} \begin{array}{l} 0 \\ 1 \end{array} \quad \text{LSB} = 0 \text{ Means [even]}$$

$$(11) \rightarrow \begin{array}{r} 101 \\ \times 000 \\ \hline 000 \end{array} \begin{array}{l} 1 \\ 1 \end{array} \quad \text{LSB} = 1 \text{ Means [odd]}$$

So, we are simply doing

$$\begin{aligned} n \& 1 &= 0 \text{ [even]} \\ &= 1 \text{ [odd]} \end{aligned}$$

Q1. Check n is odd or even.

Q2. Swapping numbers without taking 3rd variable.

$$\begin{array}{rcl}
 a \rightarrow 13 & \Rightarrow & 1101 \\
 b \rightarrow 11 & \Rightarrow & 1011 \\
 a = a \wedge b & \Rightarrow & 0110 \\
 b = a \wedge b & \Rightarrow & 1101 \\
 a = a \wedge b & \Rightarrow & 1011
 \end{array}$$

The diagram shows the iterative calculation of the bitwise AND of two numbers, 13 and 11. The first row shows a = 13 (1101) and b = 11 (1011). The second row shows the result of a AND b, which is 0110. The third row shows the result of b AND b, which is 1101. The fourth row shows the result of a AND b, which is 1011. Arrows indicate the flow of values between the two expressions.

XOR Swapping:-

$$a = 2$$

$$b = 9$$

$$\begin{array}{r} \text{0} \\ \hline \text{31} \end{array} \quad \dots \quad 0 \ 0 \ 0 \ 0 \ 1 \ 0$$
$$0 \quad \dots \quad 0 \ 0 \ 1 \ 0 \ 0 \ 1$$

$$a = 0010$$

$$b = 1001$$

$$a = a \wedge b = 1011$$

$$b = a \wedge b = 0010$$

$$a = a \wedge b = 1001$$

Bit Manipulation - II

"Tips & Tricks"

Q1) Check if k th bit is set or not?

Set $\rightarrow 1$

Unset $\rightarrow 0$

$$\begin{array}{r|l}
 13 \Rightarrow 1101 & k=2 \\
 \& \underline{0100} & \\
 \underline{0100} &
 \end{array}$$

$[(1 \ll k) \& n] > 0 \rightarrow$ ~~bit unset~~ bit is set
 else bit is unset

Q2 Set a bit

$k=2$

$$\begin{array}{r}
 \Rightarrow 16 \rightarrow 10000 \\
 \quad \underline{00100} \\
 10100 \Rightarrow 20
 \end{array}$$

$$((1 \ll k) | num) = \underline{ans}$$

Q3. Unset a bit

$k=2$

$$\begin{array}{r} 13 \Rightarrow 1101 \\ \& 1011 \\ \hline 1001 \end{array}$$

Unset 2nd bit from right

So how can we make this number
we used to do left shift $1 \ll k$ So it gives us

0100

So if we do (not) of it, we get

$$\sim(0100) \Rightarrow 1011$$

So, we will do

$$\sim(1 \ll k) \& m$$

$$m = \boxed{13} \quad k=2 \Rightarrow \boxed{9}$$

$$1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$8 + 4 + 2 + 1 = 15$$

Q4 Toggle bit (flip $0 \rightarrow 1$ & $1 \rightarrow 0$)

$$13 \rightarrow 10101$$

$$\wedge 0010$$

$$1111 = 15$$

$$1 \wedge 0 = 1$$

$$0 \wedge 0 = 0$$

$$0 \wedge 1 \rightarrow 1$$

$$1 \wedge 1 \rightarrow 0$$

$$K \rightarrow 1$$

$$13 \rightarrow$$

$$1101$$

$$0100$$

$$1001 \Rightarrow 9$$

$$(1 \leq K) \wedge \text{num}$$

$$(1 \leq K) \wedge \text{num}$$

Q5 Count set bit & (Brian Kernighan Algorithm)

$$\Rightarrow 13 \rightarrow 1101 \Rightarrow 3$$

Integer bitCount(num)

$n \& n-1 = \text{result} \rightarrow \text{unset of Right Most set bit}$

$$\begin{array}{lcl} \Rightarrow 13 & \rightarrow & 1101 \quad (n) \\ \& 12 & \rightarrow \underline{1100} \quad (n-1) \\ & 12 & \rightarrow 1100 \quad (n) \\ \& 11 & \rightarrow \underline{1011} \quad (n-1) \\ & 8 & \rightarrow 1000 \quad (n) \\ \& 7 & \rightarrow \underline{0111} \quad (n-1) \\ & 0 & \rightarrow 0000 \rightarrow \text{stop} \end{array}$$

```
int count = 0
while (num != 0) {
    num = num & (num - 1);
    count++;
}
return count
```

Q6. What is rightmost set bit?

$$\begin{array}{rcl}
 n = 12 \rightarrow 1100 & \xrightarrow{\textcircled{1}} & \text{2's complement} \\
 \& 0100 & \xleftarrow{\textcircled{2}} \\
 \hline
 0100 \Rightarrow 4 & &
 \end{array}
 \qquad
 \begin{array}{r}
 0011 \\
 +1 \\
 \hline
 0100
 \end{array}$$

• two solutions

$$\boxed{n \& -n}$$

$$n \& (\sim n + 1)$$

✓ Check if number is power of 2

$$\Rightarrow 16 \Rightarrow 10000$$

$$15 \Rightarrow 01111$$

We do set the right most bit as unset if the result is $= 0$
 $(n \& n - 1) == 0$ then it's power of 2

$$97 - 65 = 32$$

(65) A $\longrightarrow$ a(97)

(66) B $\longrightarrow$ b(98)

64 32 16 8 4 2 1
1 0 0 0 0 0 0

64 32 16 8 4 2 1
1 0 0 0 0 0 1 (64+1)=65 A
(1) or 0 1 0 0 0 0 0

1 1 0 0 0 0 1 (97) a

(char) (num | (1 << 5)) for lower case (set bit (1))

(char) (num & ~ (1 << 5)) for upper case (unset bit (0))

(char) (num ^ (1 << 5)) upper $\rightarrow$ lower

lower $\rightarrow$ upper

1 << 5 $\rightarrow$ ' ' space

~ (1 << 5) $\rightarrow$ '_'